

DEEP LEARNING

3-Week Mastery Plan

From a Single Neuron to a Deployed Generative AI App

3 Weeks	12h Total Study	8 Notebooks	7 Core Truths	4 Projects
-------------------	---------------------------	-----------------------	-------------------------	----------------------

numpy → torch → gradio → huggingface



Table of Contents

The 7 Curriculum Truths

- Truth #1 — Universal Approximation Theorem
- Truth #2 — ReLU & Vanishing Gradients
- Truth #3 — Overparameterization
- Truth #4 — Networks as Loss Minimizers
- Truth #5 — Backpropagation = Chain Rule
- Truth #6 — Loss Landscape Geometry
- Truth #7 — Capacity vs Performance

Week 1 — Build a Brain from Scratch (Pure Python)

- W1_01 — Single Neuron
- W1_02 — Backpropagation
- W1_03 — Multi-Layer Network

Week 2 — Go Deep with PyTorch

- W2_01 — PyTorch First Network
- W2_02 — Going Deeper

Week 3 — Generate & Ship

- W3_01 — Autoencoder & VAE from Scratch
- W3_02 — PyTorch Generative AI (VAE)
- W3_03 — Deploy with Gradio

Reference Material

- Cheat Sheet — Every Key Pattern
- Quick Reference Card
- Projects Ladder & Validation Checklist
- Best Resources

The 7 Curriculum Truths

These seven insights form the backbone of the entire curriculum. Every experiment, notebook, and project is designed to make you feel these truths in your bones — not just memorize them.

1

Universal Approximation Theorem

A single hidden layer with enough neurons can approximate any continuous function to arbitrary precision. This is WHY neural networks work as general-purpose function approximators. Width gives power; depth gives efficiency. You will prove this empirically in W1-03 by watching a 2-layer net learn XOR — something a single neuron cannot do.

2

ReLU & Vanishing Gradients

Sigmoid/tanh squash outputs to $[0,1]$ or $[-1,1]$. After 5 layers, gradients are multiplied together: $0.25^5 = 0.001$. The network stops learning. ReLU = $\max(0,x)$ has gradient exactly 1 for positive inputs — no squashing, no vanishing. This single insight unlocked deep learning in 2011 and is why ResNets with 152 layers are trainable.

3

Overparameterization

Classical statistics says: more parameters than data = overfitting. Deep learning violates this rule. Modern networks with billions of parameters generalize beautifully. This is 'benign overfitting' — the loss landscape of overparameterized nets has abundant flat minima that generalize well. You explore this in W2-02.

4

Networks as Loss Minimizers

Training is geometry. You are navigating a high-dimensional surface (the loss landscape) looking for valleys. Gradient descent takes steps in the direction of steepest descent. The loss function IS the objective — choose it well. MSE pushes toward means; BCE forces calibrated probabilities; KL divergence aligns distributions.

5

Backpropagation = Chain Rule

Backprop is not a mysterious algorithm — it is the chain rule from first-year calculus applied recursively. $dL/dw = dL/da * da/dz * dz/dw$. Every partial derivative in the network can be computed from the output backward using this rule. You build this from scratch in W1-02 with no library help.

8

Loss Landscape Geometry

Not all minima are equal. Sharp minima (narrow valleys) often correspond to overfitting; flat minima generalize better. Local minima are rarely the problem — the landscape is full of saddle points, and modern optimizers (Adam, SGD with momentum) escape them effectively. BatchNorm smooths the landscape; large learning rates find flatter regions.

9

Capacity vs Performance

Model capacity = how complex a function it can represent. More capacity is not always better: a 10-layer net for a 2D binary classification problem wastes capacity and creates optimization difficulty. The right model must match task complexity. Regularization (dropout, weight decay) trades capacity for generalization. This is the bias-variance tradeoff in practice.

WEEK 1 — Build a Brain from Scratch | Pure Python & NumPy | ~4 hours

Week 1 builds every concept from raw Python and NumPy — no PyTorch, no autograd, no magic. The goal is to feel backpropagation in your hands before handing it to a framework. You will implement AND gates, discover that sigmoid networks can't solve XOR, implement gradient descent by hand, and finally fix XOR with a hidden layer.

W1_01 — Single Neuron

Start here. A single neuron is a linear function followed by an activation — that's it. You implement forward pass, sigmoid activation, binary cross-entropy loss, and a manual gradient update. The AND gate trains to 100% accuracy. The XOR gate fails — and that failure is the entire motivation for everything that follows.

Key Concepts

- Forward pass: $z = w \cdot x + b$, then $a = \text{sigmoid}(z)$
- Binary cross-entropy: $L = -[y \cdot \log(a) + (1-y) \cdot \log(1-a)]$
- Gradient: $\partial L / \partial w = x \cdot (a - y)$, $\partial L / \partial b = (a - y)$
- Training loop: update w and b via gradient descent
- AND gate: converges perfectly; XOR: cannot solve (not linearly separable)
- Truth #1 in action: single neuron is too limited

Core Code Pattern

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def bce_loss(a, y):
    return -np.mean(y * np.log(a + 1e-8) + (1-y) * np.log(1-a + 1e-8))

# Single neuron forward pass
W = np.random.randn(2) * 0.01 # weights
b = 0.0 # bias

for epoch in range(1000):
    # Forward
    z = np.dot(X, W) + b # (N,)
    a = sigmoid(z) # (N,)
    loss = bce_loss(a, y) # scalar

    # Backward (chain rule by hand)
    dL_da = (a - y) / len(y) #  $\partial L / \partial a$ 
```

```

dW = X.T @ dL_da #  $\partial L / \partial W$ 
db = dL_da.sum() #  $\partial L / \partial b$ 

# Update
W -= 0.1 * dW
b -= 0.1 * db

```

W1_02 — Backpropagation

The chain rule made concrete. You derive and implement full backpropagation for a multi-layer network from scratch, including gradient checking (numerical vs analytical). Gradient checking is your unit test for backprop — if it passes to $1e-5$ tolerance, your implementation is correct.

Key Concepts

- Chain rule: $dL/dW = dL/da \cdot da/dz \cdot dz/dW$ — computed layer by layer
- Cache forward-pass values (z , a) for reuse in backward pass
- Gradient checking: $\approx (L(w+\epsilon) - L(w-\epsilon)) / 2\epsilon$ vs analytical gradient
- ReLU backward: gradient is 1 where $z > 0$, else 0 (no vanishing!)
- Truth #5: backprop is pure calculus, no magic

Core Code Pattern

```

def relu(z): return np.maximum(0, z)
def relu_prime(z): return (z > 0).astype(float)

# Forward pass through 2-layer network
def forward(X, W1, b1, W2, b2):
    z1 = X @ W1 + b1 # hidden pre-activation
    a1 = relu(z1) # hidden activation
    z2 = a1 @ W2 + b2 # output pre-activation
    a2 = sigmoid(z2) # output
    cache = (X, z1, a1, z2, a2)
    return a2, cache

# Backward pass — chain rule all the way
def backward(y, cache, W2):
    X, z1, a1, z2, a2 = cache
    dz2 = (a2 - y) / len(y) #  $dL/dz2$ 
    dW2 = a1.T @ dz2 #  $dL/dW2$ 
    db2 = dz2.sum(axis=0)
    da1 = dz2 @ W2.T #  $dL/da1$ 
    dz1 = da1 * relu_prime(z1) #  $dL/dz1$ 
    dW1 = X.T @ dz1 #  $dL/dW1$ 

```

```
db1 = dz1.sum(axis=0)
return dw1, db1, dw2, db2
```

W1_03 — Multi-Layer Network

Everything comes together. A 3-layer network solves XOR perfectly. You compare depth vs width, train on real datasets (sklearn moons/blobs), and observe the Universal Approximation Theorem experimentally — watching a network carve a non-linear decision boundary that no single neuron could ever draw.

Key Concepts

- Architecture: [2, 8, 8, 1] — input, two hidden, output
- Depth allows non-linear decision boundaries; width increases capacity
- Training on sklearn.datasets.make_moons and make_blobs
- Accuracy tracking; learning rate schedules
- Truth #1 (UAT) + Truth #2 (ReLU) + Truth #5 (backprop) all active

Core Code Pattern

```
# Full 3-layer network class
class MLP:
    def __init__(self, sizes):
        self.W = [np.random.randn(a,b)*np.sqrt(2/a)
                   for a,b in zip(sizes[:-1], sizes[1:])]
        self.b = [np.zeros(b) for b in sizes[1:]]

    def __call__(self, X):
        a = X
        self.cache = [X]
        for W,b in zip(self.W[:-1], self.b[:-1]):
            a = relu(a @ W + b)
            self.cache.append(a)
        out = sigmoid(a @ self.W[-1] + self.b[-1])
        return out

model = MLP([2, 8, 8, 1])
# Trains to >99% accuracy on XOR – impossible with 1 layer
```

WEEK 2 — Go Deep with PyTorch | Frameworks & Depth | ~4 hours

Week 2 rebuilds everything from Week 1 in PyTorch — and then goes much deeper. You learn `nn.Module`, `autograd`, `DataLoader`, and battle-tested training patterns. Then you explore the deep-learning specific challenges: initialization, batch normalization, dropout, overparameterization, and loss landscape geometry.

W2_01 — PyTorch First Network

Rebuild the Week 1 XOR solver and MNIST classifier inside PyTorch. The goal is to map every concept from raw NumPy to the equivalent PyTorch abstraction: forward pass → `model(x)`, manual gradient → `.backward()`, weight update → `optimizer.step()`. You compare SGD, Adam, and RMSprop on identical architectures.

Key Concepts

- `nn.Module`: define `forward()`, parameters tracked automatically
- `torch.autograd`: `.backward()` runs backprop; `.zero_grad()` clears accumulated gradients
- `DataLoader`: batch sampling, shuffling, parallel data loading
- Optimizer comparison: SGD (simple), Adam (adaptive LR), RMSprop (RMS scaling)
- Training loop template: the same structure used in every project
- Truth #5: `autograd` implements the same chain rule — just automatically

Core Code Pattern

```
import torch, torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(784, 256), nn.ReLU(),
            nn.Linear(256, 128), nn.ReLU(),
            nn.Linear(128, 10)
        )
    def forward(self, x):
        return self.net(x)

model = Net()
optim = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()
```

```

for epoch in range(20):
    for x, y in loader:
        loss = loss_fn(model(x), y)
        optim.zero_grad() # clear old gradients
        loss.backward() # backprop
        optim.step() # update weights

```

W2_02 — Going Deeper

Now you go truly deep. This notebook explores four pillars that make deep networks trainable in practice: proper initialization, batch normalization, dropout, and understanding the loss landscape. You also run the overparameterization experiment: train a network with more parameters than training samples and watch it still generalize.

Key Concepts

- He initialization: $\text{std} = \sqrt{2/n_{\text{in}}}$ — correct for ReLU networks
- Xavier initialization: $\text{std} = \sqrt{2/(n_{\text{in}} + n_{\text{out}})}$ — for tanh/sigmoid
- BatchNorm: normalizes each mini-batch to mean=0, std=1; learnable scale/shift
- Dropout: randomly zeros activations at rate p during training; disabled at test
- Overparameterization experiment: 100k params, 1k samples — still generalizes
- Loss landscape: wider networks have flatter, more navigable landscapes
- Truths #3 (overparam), #4 (loss minimizer), #6 (landscape geometry)

Core Code Pattern

```

# He initialization (built-in via gain parameter)
def init_weights(m):
    if isinstance(m, nn.Linear):
        nn.init.kaiming_normal_(m.weight, nonlinearity='relu')
        nn.init.zeros_(m.bias)

model.apply(init_weights)

# Network with BatchNorm + Dropout
class DeepNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(784, 512), nn.BatchNorm1d(512), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(512, 256), nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(256, 128), nn.BatchNorm1d(128), nn.ReLU(),

```

```
nn.Linear(128, 10)
)
def forward(self, x):
    return self.net(x)
# Overparameterization test
# Train on 1000 samples with a 100k-parameter network
# Result: still achieves good test accuracy – benign overfitting
```

WEEK 3 — Generate & Ship | VAE + Gradio + HuggingFace | ~4 hours

Week 3 is where everything comes together into a real product. You build a Variational Autoencoder — a generative model that learns a compressed latent space and can generate new images by sampling from it. Then you wrap it in a Gradio UI and deploy it to the world on Hugging Face Spaces.

W3_01 — Autoencoder & VAE from Scratch

Before PyTorch VAEs, build one in pure NumPy. Start with a standard autoencoder (encode → compress → decode), then add the VAE extensions: reparameterization trick, KL divergence loss, and sampling from the learned latent space. Visualize MNIST's 2D latent space to see the learned structure.

Key Concepts

- Autoencoder: Encoder maps $x \rightarrow z$ (compressed code); Decoder maps $z \rightarrow \hat{x}$
- Reconstruction loss: MSE between input x and output $\hat{x}$
- VAE extension: Encoder outputs μ and $\log(\sigma^2)$ — not a single point, a distribution
- Reparameterization trick: $z = \mu + \sigma \cdot \epsilon$ where $\epsilon \sim N(0,1)$ — keeps gradients flowing
- KL divergence: $KL(q(z|x) || p(z)) = -0.5 \cdot \sum(1 + \log(\sigma^2) - \mu^2 - \sigma^2)$
- Total VAE loss: Reconstruction Loss + $\beta \cdot$ KL Divergence
- 2D latent space visualization: digits cluster by class

Core Code Pattern

```
# VAE forward pass (pure NumPy)
def encode(X, We, be):
    h = relu(X @ We[0] + be[0]) # encoder hidden
    mu = h @ We[1] + be[1] # mean
    log_var = h @ We[2] + be[2] # log variance
    return mu, log_var

def reparameterize(mu, log_var):
    eps = np.random.randn(*mu.shape) # sample from N(0,1)
    sigma = np.exp(0.5 * log_var) # std dev
    return mu + sigma * eps # z = mu + sigma * eps

def decode(z, Wd, bd):
    h = relu(z @ Wd[0] + bd[0])
    return sigmoid(h @ Wd[1] + bd[1])

# VAE loss = reconstruction + KL
def vae_loss(X, X_hat, mu, log_var, beta=1.0):
```

```
recon = np.mean((X - X_hat)**2) # MSE reconstruction
kl = -0.5 * np.mean(1 + log_var - mu**2 - np.exp(log_var))
return recon + beta * kl
```

W3_02 — PyTorch Generative AI (VAE)

Now rebuild the VAE in PyTorch with convolutional layers for image data. Train on MNIST and/or CIFAR-10. Explore the latent space: interpolate between two digits, do latent arithmetic (digit 7 - 1 + 4 ≈ digit 4?), and generate entirely new images by sampling random z vectors.

Key Concepts

- ConvVAE: Conv2d encoder → flatten → Linear μ/σ → Linear → ConvTranspose2d decoder
- KLANnealing: start with $\beta=0$, linearly increase to 1 to prevent posterior collapse
- Latent space interpolation: smoothly morph between two digits via $z = \alpha \cdot z_1 + (1-\alpha) \cdot z_2$
- Generation: sample $z \sim N(0, I)$, decode → new image
- ELBO: Evidence Lower Bound = $E[\log p(x|z)] - KL(q(z|x) || p(z))$

Core Code Pattern

```
class ConvVAE(nn.Module):
    def __init__(self, latent_dim=16):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 32, 4, 2, 1), nn.ReLU(), # 28->14
            nn.Conv2d(32, 64, 4, 2, 1), nn.ReLU(), # 14->7
            nn.Flatten()
        )
        self.fc_mu = nn.Linear(64*7*7, latent_dim)
        self.fc_var = nn.Linear(64*7*7, latent_dim)
        self.decoder_input = nn.Linear(latent_dim, 64*7*7)
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(64, 32, 4, 2, 1), nn.ReLU(),
            nn.ConvTranspose2d(32, 1, 4, 2, 1), nn.Sigmoid()
        )

    def encode(self, x):
        h = self.encoder(x)
        return self.fc_mu(h), self.fc_var(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        return mu + std * torch.randn_like(std)
```

```
def forward(self, x):
    mu, logvar = self.encode(x)
    z = self.reparameterize(mu, logvar)
    h = self.decoder_input(z).view(-1, 64, 7, 7)
    return self.decoder(h), mu, logvar
```

W3_03 — Deploy with Gradio

Your VAE is now a product. Wrap it in a Gradio interface with controls for latent dimensions, temperature, and class conditioning. Deploy to Hugging Face Spaces with a single git push — free hosting, public URL, shareable with anyone.

Key Concepts

- Gradio: `gr.Interface()` or `gr.Blocks()` wraps any Python function in a web UI
- Model loading: `torch.load()` + `model.eval()` + `torch.no_grad()`
- Gradio components: `gr.Slider` (temperature), `gr.Image` (output), `gr.Button`
- HuggingFace Spaces: free GPU hosting, git-based deployment
- `requirements.txt` + `app.py`: all that's needed for a live deployment

Core Code Pattern

```
import gradio as gr
import torch

model = ConvVAE(latent_dim=16)
model.load_state_dict(torch.load('vae_mnist.pth', map_location='cpu'))
model.eval()

def generate(temperature: float, seed: int):
    torch.manual_seed(seed)
    with torch.no_grad():
        z = torch.randn(1, 16) * temperature
        h = model.decoder_input(z).view(1, 64, 7, 7)
        img = model.decoder(h)[0, 0].numpy() # (28, 28)
    return img

demo = gr.Interface(
    fn=generate,
    inputs=[
        gr.Slider(0.1, 2.0, value=1.0, label="Temperature"),
        gr.Slider(0, 999, value=42, step=1, label="Seed"),
    ],
    outputs=gr.Image(label="Generated Digit"),
```

```
    title="VAE Digit Generator",  
)  
demo.launch() # local; share=True for public tunnel  
# Deploy: huggingface-cli login + git push to HF Space
```

Cheat Sheet — Every Key Pattern

The 7 Truths — One Line Each

```
#1 UAT: Wide nets approximate any function
#2 ReLU:  $\max(0, x)$  solves vanishing gradients
#3 Overfit: More params still generalizes (benign)
#4 Loss: Training = minimizing a surface
#5 Backprop: Chain rule, computed backward
#6 Landscape: Flat minima = better generalization
#7 Capacity: Match model size to task complexity
```

Essential Activation Functions

```
sigmoid(z) =  $1/(1+e^{-z})$  range  $[0, 1]$  output only
tanh(z) =  $(e^z - e^{-z}) / (e^z + e^{-z})$  range  $[-1, 1]$  rarely used
relu(z) =  $\max(0, z)$  range  $[0, \infty)$  standard hidden
leaky_relu =  $\max(0.01z, z)$  prevents dying ReLU
gelu(z)  $\approx z \cdot \Phi(z)$  used in transformers
```

PyTorch Training Loop Template

```
model = nn.Sequential(...) # define architecture
optim = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss() # or BCELoss, MSELoss

for epoch in range(num_epochs):
    model.train()
    for x, y in train_loader:
        pred = model(x) # forward
        loss = loss_fn(pred, y) # compute loss
        optim.zero_grad() # clear gradients
        loss.backward() # backprop
        optim.step() # update
    model.eval() # disable dropout/BN
    with torch.no_grad():
        val_loss = loss_fn(model(x_val), y_val)
```

Initialization Guide

```
ReLU networks: nn.init.kaiming_normal_(w, nonlinearity='relu')
Sigmoid/tanh: nn.init.xavier_normal_(w)
Bias: nn.init.zeros_(b)
```

```
Bad init: nn.init.constant_(w, 0) # all neurons same gradient!
```

Optimizer Cheat Sheet

```
SGD: lr=0.01-0.1; momentum=0.9; needs tuning but generalizes well
```

```
Adam: lr=1e-3; adaptive per-param LR; fast convergence; default choice
```

```
RMSprop: lr=1e-3; adaptive; good for RNNs; similar to Adam
```

```
AdamW: Adam + weight decay decoupled; preferred for large models
```

VAE Loss Formula

```
ELBO =  $E[\log p(x|z)] - \beta * KL(q(z|x) || p(z))$ 
```

```
Reconstruction loss = MSE(x, x_hat) or BCE(x, x_hat)
```

```
KL divergence =  $-0.5 * \sum(1 + \log\_var - \mu^2 - \exp(\log\_var))$ 
```

```
 $\beta < 1$ : more reconstruction focus (sharper images)
```

```
 $\beta > 1$ : more disentanglement (smoother latent space)
```

Projects Ladder & Validation Checklist

P1	Classify with a Multi-Layer Network Train a 3-layer net on MNIST to 95%+ accuracy in pure NumPy. <i>Proves: backprop, forward pass, hyperparameter tuning</i>	Week 1
P2	Loss Landscape Explorer Visualize how loss changes with architecture and initialization choices. <i>Proves: optimization geometry, overparameterization intuition</i>	Week 2
P3	Generative Image Model Train a VAE to generate MNIST digits from latent noise. <i>Proves: VAE theory — reparameterization, KL loss, decoder</i>	Week 3
P4	Deployed Gradio App Ship P3 as a live web app on Hugging Face Spaces. <i>Proves: end-to-end — build AND deploy a real generative AI product</i>	Week 3

Validation Checklist

After Week 1 ■ Re-implement backprop without notes ■ Explain chain rule aloud ■ XOR solved with custom network ■ Gradient check passes $< 1e-5$	After Week 2 ■ PyTorch net matches NumPy accuracy ■ Loss landscape for 2 architectures ■ Explain why overparameterization works ■ Compare 3 optimizers	After Week 3 ■ VAE generates recognizable images ■ Live app on HF Spaces ■ Latent space interpolation working ■ Explain KL divergence intuitively
--	---	--

Best Resources

Category	Resource	Why It's Essential
Foundation	3Blue1Brown Neural Networks (YouTube)	Best visual intuition for backprop & gradient descent
Top-Down	fast.ai Practical Deep Learning (free)	Results-first approach; builds intuition before theory
Math/Theory	Deep Learning Book — Goodfellow et al.	Rigorous UAT, optimization, and generative model coverage
PyTorch	PyTorch Official Tutorials (docs)	nn.Module, autograd, DataLoader with official examples
Generative	Lil'Log — Understanding VAEs	Best written deep-dive on VAE math and reparameterization trick
Research	Arxiv Sanity Preserver / Papers With Code	Stay current; link papers to working code
Deployment	Gradio + HuggingFace Docs	Fastest path from trained model to shareable web app

Communities

fast.ai Forums — active, beginner-friendly

PyTorch Discuss (discuss.pytorch.org) — official community

r/MachineLearning — research discussions

HuggingFace Discord — deployment, transformers, spaces

Build. Break. Understand. Repeat. — The only way to really learn deep learning is to build the damn thing yourself. Start with W1_01 today.